Wilo-Pumpe Teil 3: PIO misst alles

Hardware statt Software — wie der RP2040-PIO HIGH- und LOW-Zeit direkt in Hardware misst und warum 250 kHz dabei Routine sind

make Magazin · Projekt-Doku

April 2026 `github.com/gerontec/wilo_pwm`

Teil 1 nutzte C-Interrupts, Teil 2 Core 1-Polling. Teil 3 delegiert die gesamte Messung an den Hardware-PIO — kein CPU-Overhead, kein Debounce, keine Firmware-Aenderung noetig. Ergebnis: stabile Pulsmessung bis 250 kHz als reines Python-Modul.

---

1 Warum PIO besser als Core-1-Polling ist

In Teil 2 lief eine enge Polling-Schleife auf Core 1 mit 26 Millionen Iterationen pro Sekunde. Das funktioniert, hat aber zwei Schwaechen:

- **Software-Debounce noetig.** Jeder Glitch auf der Leitung muss manuell gefiltert werden — mit adaptivem Schwellwert, der bei Frequenzaenderungen neu berechnet wird.
- **Firmware-Build erforderlich.** Das Core-1-Modul ist als `USER_C_MODULES` in die Firmware gelinkt — jede Aenderung erfordert einen neuen Build und BOOTSEL-Flash.

Der RP2040-PIO loest beide Probleme auf einmal. PIO (Programmable I/O) ist ein eigenstaendiger Prozessor mit eigenem Instruktionssatz und eigenem Takt, voellig unabhaengig von ARM-Core 0 und Core 1. Ein PIO-Programm laeuft deterministisch Zyklus fuer Zyklus, wird nicht von GC-Pausen unterbrochen und braucht keine IRQ-Vektoren. Und das Beste: In MicroPython laesst sich PIO als reines Python-Modul deployen — kein Compiler, kein Flash.

2 Das PIO-Assembler-Programm

Das Messprinzip ist direkt:

1. Auf steigende Flanke warten (`wait`)
2. Abwaertszaehler X von `0xFFFFFFFF` herunterzaehlen solange der Pin HIGH ist
3. Zaehlerstand in RX-FIFO schieben (`push`)
4. Dasselbe fuer die LOW-Phase wiederholen

Listing 1: pwmfeedback_pio.py — PIO-Programm fuer HIGH/LOW-Messung

```
@rp2.asm_pio()
def _pwm_measure():
    pull()            # Y-Initialwert (0xFFFFFFFF) aus TX-FIFO laden
    mov(y, osr)
```

```python

    wrap_target()

    wait(0, pin, 0)  # Sync: warten bis Pin LOW
    wait(1, pin, 0)  # warten auf steigende Flanke (Pin wird HIGH)

    # HIGH-Zeit messen: X abwaerts zaehlen solange Pin HIGH
    mov(x, y)                    # X = 0xFFFFFFFF
    label("high_loop")
    jmp(pin, "high_count")       # Pin HIGH? -> weiterzaehlen
    jmp("high_done")             # Pin LOW?  -> fertig
    label("high_count")
    jmp(x_dec, "high_loop")      # X--, und zurueck
    label("high_done")
    mov(isr, x)
    push(noblock)                # Zaehlerstand -> RX-FIFO

    # LOW-Zeit messen: X abwaerts zaehlen solange Pin LOW
    mov(x, y)
    label("low_loop")
    jmp(pin, "low_done")         # Pin HIGH? -> LOW-Phase vorbei
    jmp(x_dec, "low_loop")       # X--, weiterzaehlen
    label("low_done")
    mov(isr, x)
    push(noblock)

    wrap()
```

Das Programm besteht aus 15 Instruktionen — weit unter dem Limit von 32. Die Zaehlschleife benoetigt **2 Zyklen pro Schritt**. Bei einem PIO-Takt von 1 MHz ergibt das eine Zeitaufloesung von 2 µs.

3 State Machine und Python-Auswertung

Die State Machine wird mit einem einzigen `put()` initialisiert, der den Startwert 0xFFFFFFFF in den TX-FIFO legt:

Listing 2: Initialisierung der State Machine

```python
pin = Pin(5, Pin.IN, Pin.PULL_UP)
sm = rp2.StateMachine(
    0, _pwm_measure,
    freq=1_000_000,     # 1 MHz -> 1 us/Zyklus, 2 us/Zaehlschritt
    in_base=pin,
    jmp_pin=pin,
)
sm.put(0xFFFFFFFF)      # Y-Initialwert
sm.active(1)
```

Die Python-Seite liest HIGH- und LOW-Wert paarweise aus dem FIFO und rechnet Zeitdauer, Frequenz und Duty-Cycle:

Listing 3: Auswertung: Frequenz und Duty-Cycle aus Zaehlerstand

```python
while sm.rx_fifo() >= 2:
    x_high = sm.get()
    x_low  = sm.get()
    # (0xFFFFFFFF - x) * 2 us/Count = Pulsdauer in Mikrosekunden
```

```
    high_us = (0xFFFFFFFF - x_high) * 2
    low_us  = (0xFFFFFFFF - x_low)  * 2
    T       = high_us + low_us
    freq    = 1_000_000.0 / T         # Hz
    duty    = high_us / T * 100.0     # Prozent
```

`push(noblock)` sorgt dafuer, dass veraltete Werte verworfen werden, wenn der FIFO voll ist — Python kann die Daten im eigenen Rhythmus (alle 5 s) abholen, ohne dass die State Machine blockiert.

4 Frequenzlimit: Wann stoesst PIO an seine Grenzen?

Die Zaehlschleife benoetigt 2 PIO-Zyklen pro Schritt. Bei einem PIO-Takt von 1 MHz gilt:

$$t_{\min} = 2 \times \frac{1}{f_{PIO}} = 2\,\mu s$$

Fuer ein Signal mit 50 % Duty-Cycle muss sowohl HIGH als auch LOW mindestens 2 µs betragen — also Periode $\geq$ 4 µs, Frequenz $\leq$ **250 kHz**.

Bei niedrigem Duty-Cycle (z. B. 5 %) ist der HIGH-Impuls kurz: $0{,}05 \times T$. Das Minimum von 2 µs HIGH-Dauer limitiert dann die Frequenz: $f_{\max} = 1/(2\,\mu s/0{,}05) =$ **25 kHz**.

Wer hoehere Frequenzen braucht, erhoet den PIO-Takt auf bis zu 125 MHz (Systemtakt):

```
sm = rp2.StateMachine(0, _pwm_measure, freq=125_000_000, ...)
# -> 16 ns/Zyklus, 32 ns/Zaehlschritt -> max ~31 MHz bei 50% Duty
```

PIO-Takt	Aufloesung	Max. Freq. (50% Duty)
1 MHz	2 µs	250 kHz
10 MHz	200 ns	2,5 MHz
125 MHz	16 ns	~31 MHz

Fuer 75 Hz (Wilo-Pumpe) ist der 1-MHz-Takt 3000-fach ueberdimensioniert — die Messung ist absolut stabil.

5 Vergleich aller vier Methoden

Methode	Max. Freq.	Debounce	CPU-Last	Build
Python IRQ (pure)	~2 kHz	noetig	hoch	nein
C-Natmod IRQ	~10 kHz	noetig	mittel	.mpy
Core 1 Polling	~250 kHz	adaptiv	Core 1	Firmware
PIO @ 1 MHz	250 kHz	nicht noetig	null	nein
PIO @ 125 MHz	~31 MHz	nicht noetig	null	nein

PIO ist in allen Kategorien besser als die Software-Ansaetze — und benoetigt als einzige Methode weder einen Firmware-Build noch einen BOOTSEL-Flash.

6 Drop-in-Ersatz ohne Aenderung an main.py

Das Modul `pwmfeedback_pio.py` implementiert dieselbe API wie alle Vorgaenger:

Listing 4: Umschalten in main.py: eine Zeile

```
# vorher:
import pwmfeedback_core1 as pwmfeedback    # Core 1 Polling
# nachher:
import pwmfeedback_pio as pwmfeedback      # PIO Hardware
```

`init_feedback_pin()` startet die State Machine, `get_pump_feedback()` liefert dasselbe Dict mit `PIN5_Freq_Hz`, `PumpDuty` und `PumpStatus`. Der Rest von `main.py` — MQTT, Watchdog, Rampe, Boost — bleibt unveraendert.

Deploy per WebREPL oder mpremote, kein BOOTSEL noetig:

Listing 5: Deploy ohne Firmware-Flash

```
mpremote connect /dev/ttyACM0 \
    cp pwmfeedback_pio.py :pwmfeedback_pio.py + \
    cp main.py :main.py + reset
# -> sofort aktiv, keine neue Firmware
```

7 Messwerte im Betrieb

Gemessene Werte der Wilo iPWM-Pumpe im Normalbetrieb:

Kenngroesse	**Wert**
Frequenz	75,0 Hz (stabil, keine Transienten)
Duty-Cycle	29,5–42 % (lastabhaengig)
Aufloesung	2 µs
CPU-Last	0 % (reine Hardware)

Die Glitch-Probleme aus Teil 2 — kurze Transienten bei 153 Hz oder 181 Hz durch falsche Debounce-Timing — treten mit dem PIO-Ansatz nicht auf. Die `wait()`-Instruktion synchronisiert auf die echte Flanke, der Zaehler laeuft deterministisch ohne Unterbrechung.

Code, Firmware & Deploy-Script: https://github.com/gerontec/wilo_pwm